

Introduction

Objectifs : développer une application web intégrant un back office et une interface dynamique dans un contexte international.

En tant que développeur web, on doit répondre à la question : Comment développer une application ou un service web en proposant une expérience utilisateur de qualité ?

Une partie significative des contenus sera réalisée en anglais.

Proposer une application fonctionnelle en prenant en compte l'expérience utilisateur, l'ergonomie et l'accessibilité. L'application ou le service devra obligatoirement utiliser un framework front et back. L'application devra utiliser du SASS/SCSS pour toute la partie CSS. L'application sera proposée dans plusieurs langues.

Un back office correctement configuré, avec une interface et une expérience utilisateur de qualité devra être disponible (possible d'utiliser des outils comme EasyAdmin correctement configurés et personnalisés).

Le back-office pourra être en Symfony uniquement. La partie application devra obligatoirement être en vueJs.

Vous veillerez à soigner l'apparence du site et la mise en pratique de vos connaissances en Webdesign. Un logo simple sera proposé.

Vous imaginerez le contexte et le storytelling de votre entreprise/application. Cela se traduira sur la page d'accueil et de présentation du service proposé.

Socle Technique Commun (Imposé) :

Chaque projet doit respecter cette architecture :

- Backend : Symfony 7/8 (API Platform ou contrôleurs custom retournant du JSON).
- Frontend : Vue.js 3 (Composition API).
- Base de données : MySQL/MariaDb.
- Internationalisation (i18n) :
 - Backend : Gestion des contenus traduisibles en DB (via extensions Gedmo ou entités dédiées).
 - Frontend : Utilisation de vue-i18n pour l'interface.
- UX/UI : Maquettage conseillé et tests utilisateurs simples et respect des règles d'accessibilité (WCAG).

Contraintes et critères imposés :

De manière générale :

- Les sites devra être hébergé sur votre VPS
- Le site devra être sécurisé (https, ...)
- Le produit devra être accessible (RGAA AA) et "éco-conçu"
- Le produit utilisera un framework front et back

L'UX du site devra être particulièrement soignée. Le site devra être ergonomique et intuitif.

Envoyer les mails nécessaires pour ce type de site

Sujet

Sujet : Collection Monster Energy (Communauté & Chasse)

Objectif : Créer la plateforme communautaire de référence pour gérer sa collection de boissons énergisantes.

Le Concept : Une application "compagnon" mobile-first permettant aux collectionneurs de répertorier leurs canettes, de scanner des codes-barres en magasin pour vérifier leur inventaire, et de partager leurs trouvailles avec leurs amis.

Descriptif du projet L'application centralise la gestion de collection avec une forte dimension sociale et d'exploration.

- **Catalogue Collaboratif :** Base de données exhaustive incluant les variantes, les éditions arrêtées ("Discontinued") et les niveaux de rareté. Les utilisateurs peuvent proposer de nouveaux ajouts validés par un administrateur.
- **Scanner & Inventaire :** Fonctionnalité d'appareil photo pour scanner le code-barres d'une canette et savoir instantanément si elle est dans la collection (système binaire "Possédée" / "Non possédée").
- **Monster Hunter (Géolocalisation) :** Signalement des canettes dans les magasins locaux pour aider les autres membres à dénicher des éditions rares.
- **Social & Dashboard ("Le Flex") :** Profils publics/privés, statistiques graphiques de la collection (camemberts par pays/saveur), système de badges et fil d'actualité axé sur les avis et les nouvelles sorties officielles.

Focus UX & I18n

- **UX :** Identité visuelle très marquée en accord avec la marque (Dark mode obligatoire, fond noir, vert néon, typographie agressive). Ergonomie orientée "utilisation en rayon" avec de gros boutons tactiles et une navigation au pouce.
- **I18n :** Adaptation aux spécificités régionales (une même canette peut avoir un code-barres ou un design différent selon le pays d'origine) et traduction des saveurs et des types (sans sucre, café, thé).

Spécifications/pistes Techniques :

- **Base de données :** Modélisation relationnelle robuste sous MariaDB pour gérer les liaisons complexes (utilisateurs, catalogue de canettes, collections, amis, avis, signalements géolocalisés).
- **Module Scanner :** Intégration d'une librairie JavaScript de lecture de codes-barres (EAN) capable d'interroger la base de données en temps réel et de pré-remplir le formulaire d'ajout si le code est inconnu.
- **Cartographie :** Utilisation de l'API de géolocalisation couplée à une carte interactive (type Leaflet ou Mapbox) pour afficher les "Trouvailles récentes" du mode Monster Hunter.
- **I18n spécifique :** Gestion multilingue du catalogue collaboratif et localisation dynamique des statuts (ex: le badge rouge "Discontinued" doit s'adapter à la langue de l'utilisateur).

Point de vigilance UX : Le tunnel de "scan en magasin". La rapidité d'exécution est cruciale : l'utilisateur doit pouvoir sortir son téléphone, scanner la canette, et obtenir son statut (déjà possédée ou non) de manière instantanée, avec des retours visuels évidents (couleurs tranchées) même dans un supermarché mal éclairé ou avec une faible connexion internet. L'export de la collection (Data Portability) doit aussi être facilement accessible pour rassurer les gros collectionneurs.

Contexte

Vision et Objectifs

MonsterCollection est la plateforme communautaire de référence conçue par et pour les collectionneurs de boissons énergisantes *Monster Energy*. Pensée avant tout comme un outil de terrain ("Mobile First"), l'application permet à ses utilisateurs de gérer leur inventaire en temps réel, de chasser de nouvelles canettes en magasin et d'interagir avec d'autres passionnés.

Identité Visuelle et UX (Expérience Utilisateur)

L'application doit refléter l'ADN de la marque, avec une interface moderne, agressive et taillée pour l'efficacité.

- **Palette de couleurs** : Mode Sombre (Dark Mode) exclusif. Fond noir profond (#000000), typographie blanche/grise, et accents de couleur **Vert Néon** typique de la marque pour les boutons d'action (Call to Action).
- **Style** : Premium, urbain, avec une légère touche "gaming" ou "grunge".
- **Ergonomie "En Rayon"** : L'interface mobile doit être utilisable à une main (le pouce) avec une barre de navigation inférieure. Les actions fréquentes (comme le scan) doivent être accessibles en un clic depuis l'accueil.

Base de données

Les différentes tables de la base de données :

1. Table **user** (Utilisateurs)

Cette table gère l'authentification et les paramètres de confidentialité des comptes.

- **id** (INT, Clé Primaire, Auto-incrément)
- **email** (VARCHAR 180, Unique, Non Null)
- **roles** (JSON, Non Null) : Stocke les rôles de l'utilisateur (ex: ["ROLE_USER"] ou ["ROLE_ADMIN"]).
- **password** (VARCHAR 255, Non Null) : Mot de passe haché.
- **pseudo** (VARCHAR 50, Unique, Non Null)
- **avatar** (VARCHAR 255, Nullable) : Lien vers l'image de profil.
- **is_public** (BOOLEAN, Défaut : True) : Gère la confidentialité du profil.
- **created_at** (DATETIME, Non Null)

2. Table **monster** (Le Catalogue)

Cette table contient l'exhaustivité des références de canettes. Les champs textuels (saveur, type, origine) devront être gérés par l'extension Gedmo pour l'internationalisation.

- **id** (INT, Clé Primaire, Auto-incrément)
- **ean** (VARCHAR 50, Unique, Nullable) : Le code-barres pour le module de scan.
- **name** (VARCHAR 100, Non Null)
- **flavor** (VARCHAR 100, Non Null) : La saveur (ex: Mangue, Agrumes).
- **origin** (VARCHAR 50, Non Null) : Le pays d'origine.
- **type** (VARCHAR 50, Non Null) : Le type de gamme (ex: Ultra, Juice, Java).
- **release_year** (INT, Nullable) : L'année de sortie.
- **image** (VARCHAR 255, Non Null) : Lien vers la photo de la canette.
- **rarity** (VARCHAR 50, Non Null) : L'échelle de rareté (Commun, Rare, Légendaire).
- **is_discontinued** (BOOLEAN, Défaut : False) : Statut "Arrêtée".
- **estimated_value** (FLOAT, Nullable) : Valeur financière sur le marché.
- **status** (VARCHAR 20, Défaut : 'pending') : Pour la modération des ajouts utilisateurs (pending, approved, rejected).
- **submitted_by** (INT, Clé Étrangère vers **user.id**, Nullable)
- **created_at** (DATETIME, Non Null)

3. Table **ext_translations** (Traductions Gedmo)

Conformément au choix de l'extension Gedmo pour Doctrine, la base de données ne créera pas une table par langue. Gedmo utilise une table centrale générée automatiquement qui contient toutes les traductions du backend :

- **id** (INT, Clé Primaire)
- **locale** (VARCHAR 8) : La langue (ex: 'fr', 'en').
- **object_class** (VARCHAR 255) : L'entité concernée (ex: `App\Entity\Monster`).
- **field** (VARCHAR 32) : Le champ traduit (ex: **flavor**).
- **foreign_key** (VARCHAR 64) : L'ID de la canette.
- **content** (LONGTEXT) : La valeur traduite.

4. Table **collection_item** (Inventaire & Marketplace)

Il s'agit de la table pivot entre l'utilisateur et les canettes. Elle gère la possession binaire et les disponibilités pour les échanges.

- **id** (INT, Clé Primaire, Auto-incrément)
- **user_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **monster_id** (INT, Clé Étrangère vers **monster.id**, Non Null)
- **is_possessed** (BOOLEAN, Défaut : False) : L'inventaire binaire.
- **is_for_trade** (BOOLEAN, Défaut : False) : Active le statut "À échanger" pour la zone Marketplace.
- **added_at** (DATETIME, Non Null) : Permet de générer le graphique de la timeline d'acquisition.

5. Table **review** (Système d'Avis)

Stocke les notes, les critères spécifiques et les commentaires.

- **id** (INT, Clé Primaire, Auto-incrément)
- **user_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **monster_id** (INT, Clé Étrangère vers **monster.id**, Non Null)
- **global_rating** (INT, Non Null) : Note sur 10.
- **taste_rating** (INT, Nullable) : Critère Goût.
- **design_rating** (INT, Nullable) : Critère Design.
- **price_rating** (INT, Nullable) : Critère Prix.
- **comment** (TEXT, Nullable) : Zone de texte libre.
- **visibility** (VARCHAR 20, Défaut : 'public') : Gestion de la confidentialité de l'avis (privé, amis, public).
- **created_at** (DATETIME, Non Null)

6. Table **sighting** (Monster Hunter)

Enregistre les données de géolocalisation pour le signalement de canettes en magasin.

- **id** (INT, Clé Primaire, Auto-incrément)
- **user_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **monster_id** (INT, Clé Étrangère vers **monster.id**, Non Null)
- **latitude** (DECIMAL 10,8, Non Null)
- **longitude** (DECIMAL 11,8, Non Null)
- **shop_name** (VARCHAR 100, Nullable) : Nom du magasin.
- **city** (VARCHAR 100, Nullable) : Ville de la trouvaille.
- **created_at** (DATETIME, Non Null) : Date du signalement.

7. Table **friendship** (Réseau Social)

Gère les demandes et les listes d'amis entre les utilisateurs.

- **id** (INT, Clé Primaire, Auto-incrément)
- **requester_id** (INT, Clé Étrangère vers **user.id**, Non Null) : Celui qui envoie la demande.
- **receiver_id** (INT, Clé Étrangère vers **user.id**, Non Null) : Celui qui la reçoit.

- **status** (VARCHAR 20, Défaut : 'pending') : État de la relation (pending, accepted, blocked).
- **created_at** (DATETIME, Non Null)

8. Table **badge** (Gamification)

Répertorie les succès déblocables de l'application.

- **id** (INT, Clé Primaire, Auto-incrément)
- **name** (VARCHAR 100, Non Null) : Nom du badge (ex: "Globe-trotter").
- **description** (VARCHAR 255, Non Null)
- **icon** (VARCHAR 255, Non Null) : Lien vers l'image du badge.
- **criteria_code** (VARCHAR 50, Non Null) : Identifiant technique utilisé par le code Symfony pour valider l'obtention.

9. Table **user_badge** (Table Pivot Badges)

Associe les badges débloqués aux utilisateurs.

- **user_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **badge_id** (INT, Clé Étrangère vers **badge.id**, Non Null)
- **unlocked_at** (DATETIME, Non Null)
- *Clé primaire composite (user_id, badge_id)*

10. Table **message** (Messagerie Marketplace)

Permet la communication interne pour organiser les échanges de canettes.

- **id** (INT, Clé Primaire, Auto-incrément)
- **sender_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **receiver_id** (INT, Clé Étrangère vers **user.id**, Non Null)
- **content** (TEXT, Non Null)
- **is_read** (BOOLEAN, Défaut : False)
- **created_at** (DATETIME, Non Null)

Fonctionnalités

Liste des fonctionnalités à développer :

1. Gestion des Utilisateurs et Social

- Comptes : Inscription, connexion et récupération de mot de passe.
- Confidentialité : Paramétrage du profil (Public ou Privé) modifiable à tout moment.
- Réseau d'amis : Système de demande, d'acceptation et de retrait d'amis.
- Fil d'actualité : Flux dynamique affichant les nouveaux avis des amis et les sorties officielles.
- Zone d'Échange (Marketplace) :
 - Statut "À échanger" activable sur les doublons.
 - Messagerie interne basique pour organiser les échanges entre membres.
 - Filtre de recherche pour trouver des canettes à échanger à proximité.
- Consultation du "Flex Dashboard" et de la collection des autres membres (soumis aux règles de confidentialité).

2. Recherche Unifiée (Omnibox)

- Barre de recherche globale : Accès permanent en haut de l'interface.
- Autocomplétion : Suggestions en temps réel lors de la saisie.
- Segmentation des résultats : Affichage distinct des "Canettes" et des "Membres".
- Accès direct au statut personnel (possédée/non possédée) et à son propre avis depuis les résultats de recherche.
- Affichage direct du rang et de l'avatar pour les profils membres.

3. Le Catalogue Collaboratif (Base de données)

- Fiches techniques complète : Nom, saveur, image, origine, type, année, code EAN.
- Indicateurs d'état : Rareté (Commun à Légendaire) et statut "Arrêtée" (Discontinued).
- Valeur du marché : Champ "Valeur estimée moyenne" pour les canettes rares ou anciennes.
- Modération : Formulaire de proposition de nouvelles canettes et interface de validation pour les administrateurs.
- Filtres avancés : Tri par pays, type (Ultra, Juice, Java, etc.), saveur etc...

4. Gestion de la Collection Personnelle

- **Inventaire Binaire** : Bouton "Possédée" / "Non possédée" (statut négatif par défaut).
- **Listes Personnalisées** : Création de dossiers thématiques (ex: "Top 10 préférées", "Objectifs d'importation").
- **Exportation** : Génération de la liste complète au format CSV ou PDF.

5. Outils Mobiles ("Sur le Terrain")

- Module d'accès à l'appareil photo du smartphone.
- Module de Scan Avancé :
 - Scanner EAN : Lecture des codes-barres avec détection automatique.

- Détection et affichage instantané du statut de possession après un scan reconnu.
- Redirection automatique vers le formulaire d'ajout (avec EAN pré-rempli) pour un scan inconnu.
- Monster Hunter (Géolocalisation) :
- Signalement de canettes trouvées en magasin.
- Carte interactive communautaire affichant les trouvailles récentes locales.
- Alertes Locales (Push) : Notifications si une canette de la "Wishlist" est signalée à proximité (rayon de 10km).
- Mode Hors-Ligne (Offline Sync) : Consultation et scan possibles sans réseau, avec synchronisation automatique lors du retour de la connexion.

6. Système d'Avis et de Notation

- Note globale : Score sur 10 affiché sur la fiche.
- Commentaires : Zone de texte libre pour chaque avis.
- Interactions : Système de "J'aime" sur les avis laissés par les amis.
- possibilité de rendre l'avis privé, amis ou public

7. Gamification et Statistiques ("Le Flex")

- Affichage du pourcentage de la collection mondiale complétée.
- Badges : Succès déblocables selon les actions (ex: "Globe-trotter" pour 5 pays différents).
- Analyses Visuelles : Graphiques de répartition par saveur, type et origine.
- Historique d'Acquisition (Timeline) : Graphique montrant l'évolution et la croissance de la collection dans le temps.
- Valorisation de Collection : Calcul automatique de la valeur financière totale estimée de l'inventaire.
- Compteur communautaire : Statistiques sur le nombre d'avis et de signalements "Monster Hunter" effectués.

Pages

Liste des pages à développer :

1. Écrans d'Authentification (Onboarding)

- **Page de Connexion / Inscription :**
 - Formulaire classique (Email, Mot de passe, Pseudo).
 - Option "Mot de passe oublié".
 - Choix immédiat de la confidentialité : "Profil Public" ou "Profil Privé".

2. Page d'Accueil (Le Feed Social)

- **Barre de Recherche (Omnibox) :** Fixée en haut pour rechercher canettes et membres.
- **Bouton d'Action Rapide :** Un gros bouton central (ou flottant) pour lancer le Scanner instantanément.
- **Fil d'Actualité :**
 - Nouvelles sorties officielles ajoutées au catalogue.
 - Derniers avis détaillés laissés par les amis (avec bouton "J'aime").
- **Indicateur Réseau :** Une petite icône signalant si l'application est en "Mode Hors-Ligne".

3. Le Catalogue / Explorateur

- **En-tête de Filtres :** Menus déroulants pour trier par Type, Saveur, Origine, Rareté, et disponibilité (Arrêtée).
- **Grille de Canettes :** Affichage en miniatures avec :
 - Nom de la Monster.
 - Pastille de couleur (Vert = Possédée, Gris = Non possédée).
 - Badge visuel si rareté élevée ou arrêtée.
- **Bouton de Contribution :** Lien vers le formulaire "Proposer une canette non répertoriée".

4. Fiche Produit Détaillée (La Canette)

- **Informations Techniques :** Image HD, Nom, Saveur, Origine, Année, Code EAN, Rareté, Valeur estimée.
- **Barre d'Actions Utilisateur :**
 - Bouton switch : Possédée / Non possédée.
 - Bouton d'ajout aux listes : "À échanger", "Wishlist", ou listes personnalisées.
- **Zone d'Avis Personnel :**
 - Attribution de la note globale (/10) et multicritère (Goût, Design, Prix).
 - Champ de texte pour le commentaire personnel.
- **Zone "Monster Hunter" :** Bouton "Je l'ai vue en magasin" (pour géolocaliser).
- **Avis de la Communauté :** Liste des avis laissés par les amis et la communauté sur cette canette.

5. L'Outil de Scan (Caméra)

- **Vue Caméra Plein Écran** : Zone de ciblage pour le code-barres (EAN) ou la reconnaissance d'image.
- **Feedback Superposé (AR basique)** : Apparition d'une fenêtre pop-up dès le scan :
 - *Si possédée* : Cadre vert avec message "Tu l'as déjà !".
 - *Si non possédée* : Cadre bleu avec bouton "Ajouter à ma collection".
 - *Si inconnue* : Redirection vers le formulaire d'ajout pré-rempli.

6. La Carte "Monster Hunter" (Géolocalisation)

- **Carte Interactive** : Vue type Google Maps/Leaflet centrée sur la position de l'utilisateur.
- **Marqueurs de Trouvailles** : Points sur la carte indiquant où des canettes ont été signalées.
- **Liste de Proximité** : Affichage sous forme de liste des dernières trouvailles autour de soi (Magasin, Ville, Date du signalement, Nom de la canette).

7. Mon Profil ("Le Flex Dashboard")

- **En-tête Profil** : Avatar, Pseudo, et rang actuel.
- **Statistiques Globales** :
 - Pourcentage de complétion du catalogue mondial.
 - Valeur financière totale estimée de la collection.
- **Analyses Graphiques** :
 - Graphiques (camemberts ou barres) : Répartition par pays et par saveur.
 - Timeline d'acquisition (courbe d'évolution).
- **Mes Badges** : Grille des succès débloqués.
- **Mes Listes** : Accès aux dossiers thématiques créés par l'utilisateur.

8. Profil Public d'un Membre (Vue Externe)

- **Si le profil est Public (ou Ami)** : Affichage de son "Flex Dashboard", de sa collection, de ses badges, et d'un bouton "Proposer un échange" / "Retirer des amis".
- **Si le profil est Privé** : Affichage restreint (Avatar, Pseudo, Rang) avec un bouton unique "Envoyer une demande d'ami".

9. La Zone d'Échange (Marketplace)

- **Liste des Disponibilités** : Affichage des canettes marquées "À échanger" par les membres à proximité.
- **Messagerie** : Boîte de réception simple listant les conversations actives avec d'autres collectionneurs pour organiser les échanges.

10. Paramètres et Administration

- **Paramètres Utilisateur :**
 - Modification des données (Mot de passe, Email, Avatar).
 - Gestion de la confidentialité (Basculer Public/Privé).
 - Exportation des données (Bouton "Télécharger ma collection en CSV/PDF").
 - Gestion des notifications (Activer/Désactiver les alertes "Monster Hunter").
- **Espace Admin (Réservé au créateur du site) :**
 - File d'attente des propositions de canettes à valider.
 - Gestion des utilisateurs et signalements.

Composants

Liste des composants Vue.js à développer :

1. Les Layouts (Gabarits de page)

Ces composants englobent les vues pour maintenir la structure de base (navigation, en-têtes) selon que l'utilisateur est connecté ou non.

- **App.vue** : Le composant racine qui initialise le routeur et l'état global (Pinia).
- **AuthLayout.vue** : Un gabarit épuré utilisé uniquement pour les pages de connexion et d'inscription (sans menu de navigation).
- **MainLayout.vue** : Le gabarit principal pour les utilisateurs connectés. Il intègre obligatoirement la navigation au pouce pour répondre à l'ergonomie mobile-first "en rayon".
 - **NavbarBottom.vue** : La barre de navigation inférieure (Accueil, Catalogue, Scanner, Map, Profil).
 - **TopHeader.vue** : L'en-tête supérieur contenant le logo et l'accès rapide aux notifications/paramètres.

2. Les Composants UI Communs (Les "Atomes")

Ce sont des éléments d'interface génériques, stylisés avec tes variables SCSS (fond noir, typographie agressive, accents vert néon), réutilisables sur toutes les pages.

- **BaseButton.vue** : Bouton générique avec des props (variantes : **primary** néon, **secondary** gris, **danger** rouge).
- **BaseInput.vue** : Champ de saisie (texte, email, mot de passe) avec gestion des erreurs et icônes.
- **ToggleSwitch.vue** : Interrupteur stylisé (ex: pour basculer de Profil Public à Privé).
- **PossessionButton.vue** : Le bouton binaire central "Possédée / Non possédée". Il doit réagir instantanément au clic (passage au vert).
- **RarityBadge.vue** : Pastille visuelle dynamique affichant la rareté ou le statut "Arrêtée" (Discontinued). Ce composant devra intégrer **vue-i18n** pour traduire dynamiquement le statut.
- **UserAvatar.vue** : Miniature circulaire de l'image de profil d'un membre.
- **LoadingSpinner.vue** : Animation d'attente (loader) personnalisée aux couleurs de Monster.

3. Les Composants "Métier" (Les "Molécules" et "Organismes")

Ce sont des composants plus complexes, liés à une fonctionnalité spécifique de ton projet.

A. Catalogue & Recherche

- **Omnibox.vue** : La barre de recherche unifiée globale. Elle contient un **BaseInput** et gère l'autocomplétion textuelle prédictive.
- **OmniboxResults.vue** : Le menu déroulant listant les résultats, séparés entre "Canettes" et "Membres".
- **MonsterCard.vue** : La miniature d'une canette affichée dans la grille du catalogue. Elle contient l'image, le nom, le **RarityBadge** et le **PossessionButton**.
- **FilterMenu.vue** : Le menu déroulant ou la modale permettant de filtrer les canettes (Type, Saveur, Origine).

B. Feed Social & Avis

- **FeedItem.vue** : Une carte représentant une activité dans le fil d'actualité (un nouvel avis d'un ami ou une sortie officielle).
- **ReviewCard.vue** : L'affichage d'un avis utilisateur (Note /10, détails multicritères, commentaire et bouton "J'aime").
- **ReviewForm.vue** : Le formulaire permettant à l'utilisateur de laisser son avis sur une fiche produit.

C. Fonctionnalités Avancées (Terrain & Gamification)

- **BarcodeScanner.vue** : Le module intégrant la librairie de lecture de codes-barres (EAN) via la caméra du smartphone.
- **ScanResultPopup.vue** : La modale superposée (AR basique) qui affiche le résultat instantané (Cadre vert "Possédée", Cadre bleu "Ajouter").
- **MapTracker.vue** : L'intégration de la carte interactive (ex: Leaflet ou Mapbox) pour afficher les marqueurs du mode Monster Hunter.
- **StatChart.vue** : Composant "wrapper" (englobant une librairie comme Chart.js) pour afficher les camemberts de saveurs ou la timeline d'acquisition dans le "Flex Dashboard".
- **AchievementBadge.vue** : L'icône et la description d'un succès débloqué (ex: "Globe-trotter").

4. Les Vues / Pages (Dossier **views/**)

Chaque vue correspond à une route définie dans ton **vue-router**. Elles orchestrent les composants listés ci-dessus.

- **LoginView.vue / RegisterView.vue** : Écrans d'authentification.
- **HomeFeedView.vue** : Écran d'accueil avec l'Omnibox et le fil d'actualité.
- **CatalogView.vue** : L'explorateur complet avec les **FilterMenu** et une grille de **MonsterCard**.
- **MonsterDetailView.vue** : La fiche complète d'une canette.
- **ScannerView.vue** : L'outil de scan en plein écran avec la caméra.
- **HunterMapView.vue** : La page de géolocalisation pour trouver des canettes en magasin.
- **FlexDashboardView.vue** : Le profil personnel avec les graphiques de statistiques (**StatChart**) et l'exportation de la collection (Data Portability).
- **PublicProfileView.vue** : La vue du profil d'un autre membre (avec gestion de l'affichage si le profil est public ou privé).
- **MarketplaceView.vue** : La zone listant les canettes "À échanger" avec la liste des disponibilités à proximité.
- **SettingsView.vue** : Page de configuration (mot de passe, confidentialité, gestion des notifications, et un sélecteur **LanguageSwitcher.vue** pour le multilingue).

Contrat d'Interface API

1. Authentification et Utilisateurs (/api/users)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
POST	/api/login_check	Connexion. Retourne le Token JWT.	email, password	Non
POST	/api/register	Création d'un nouveau compte.	email, password, pseudo, is_public	Non
GET	/api/users/me	Récupère le profil complet et les stats du "Flex Dashboard" de l'utilisateur connecté.	Aucun	Oui
PUT	/api/users/me	Modifie les paramètres du compte (avatar, confidentialité).	pseudo, avatar, is_public	Oui
GET	/api/users	Recherche de profils membres (pour l'Omnibox).	Paramètre d'URL : ?q={recherche}	Oui
GET	/api/users/{id}	Récupère le profil public d'un autre membre.	Aucun	Oui

2. Le Catalogue des Canettes (/api/monsters)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
GET	/api/monsters	Liste le catalogue. Utilisé pour l'explorateur et la recherche Omnibox.	Paramètres d'URL : ?q={nom}, ?type={type}, ?flavor={saveur}, ?page={n}	Non
GET	/api/monsters/{id}	Récupère la fiche détaillée d'une canette spécifique.	Aucun	Non

GET	<code>/api/monsters/ean/{ean}</code>	Module Scanner : Cherche une canette via son code-barres.	Aucun	Non
POST	<code>/api/monsters</code>	Modération : Soumet une nouvelle canette pour ajout au catalogue (statut: <i>pending</i>).	<code>name, flavor, ean, origin, image_base64</code>	Oui

3. Gestion de la Collection (`/api/collections`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
POST	<code>/api/collections/toggle</code>	Bascule le statut binaire (Possédée / Non possédée) d'une canette pour l'utilisateur.	<code>monster_id</code>	Oui
PUT	<code>/api/collections/{id}/trade</code>	Active ou désactive le statut "À échanger" pour une canette possédée en double.	<code>is_for_trade</code> (booléen)	Oui
GET	<code>/api/collections/export</code>	Génère l'export de la collection (Portabilité des données).	Paramètre d'URL : <code>?format=csv</code> ou <code>pdf</code>	Oui

4. Avis et Interactions Sociales (`/api/reviews` & `/api/feed`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
GET	<code>/api/monsters/{id}/reviews</code>	Liste les avis validés pour une canette spécifique.	Aucun	Non

POST	<code>/api/reviews</code>	Ajoute ou modifie un avis sur une canette.	<code>monster_id,</code> <code>global_rating,</code> <code>taste_rating,</code> <code>design_rating,</code> <code>comment,</code> <code>visibility</code>	Oui
POST	<code>/api/reviews/{id}/like</code>	Ajoute un "J'aime" sur l'avis d'un autre utilisateur.	Aucun	Oui
GET	<code>/api/feed</code>	Récupère le fil d'actualité chronologique (nouvelles sorties + avis des amis).	Paramètre d'URL : <code>?page={n}</code>	Oui

5. Réseau d'Amis (`/api/friendships`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
POST	<code>/api/friendships/request</code>	Envoie une demande d'ami à un profil.	<code>receiver_id</code>	Oui
PUT	<code>/api/friendships/{id}/accept</code>	Accepte une demande d'ami en attente.	Aucun	Oui
DELETE	<code>/api/friendships/{id}</code>	Supprime un ami ou refuse une demande.	Aucun	Oui

6. Monster Hunter et Carte Géolocalisée (`/api/sightings`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
POST	<code>/api/sightings</code>	Signale une canette trouvée en magasin (Géolocalisation).	<code>monster_id,</code> <code>latitude,</code> <code>longitude,</code> <code>shop_name</code>	Oui

GET	<code>/api/sightings</code>	Récupère les trouvailles récentes autour d'une position pour la carte Leaflet.	Paramètres d'URL : <code>?lat={latitude},</code> <code>?lng={longitude},</code> <code>?radius={km}</code>	Oui
------------	-----------------------------	--	--	------------

7. Marketplace et Messagerie Interne (`/api/marketplace`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
GET	<code>/api/marketplace/trades</code>	Liste les canettes disponibles à l'échange autour de soi.	Paramètres d'URL : <code>?lat={latitude}</code> , <code>?lng={longitude}</code>	Oui
GET	<code>/api/messages</code>	Récupère la liste des conversations actives.	Aucun	Oui
POST	<code>/api/messages</code>	Envoie un message direct à un utilisateur pour organiser un échange.	<code>receiver_id,</code> <code>content</code>	Oui

8. Gamification (`/api/badges`)

Méthode	Route / Endpoint	Description	Payload (Body) / Paramètres	Auth. Requise
GET	<code>/api/badges</code>	Liste tous les succès/badges existants dans l'application.	Aucun	Non
GET	<code>/api/users/me/badges</code>	Liste uniquement les badges débloqués par l'utilisateur connecté avec la date d'obtention.	Aucun	Oui

Notes

livrable de fin de projet

Dossier technique (livrable fin de projet)

Le dossier technique est un document professionnel remis en fin de projet. Il devra contenir les éléments suivants :

- Résumé du projet (reprise des éléments du cahier des charges, de la demande client, ...)

- Contexte du projet (en perspective d'une agence répondant à une demande) et objectifs (fonctionnalités, ...)

- Organisation du projet et de son équipe, gestion des tâches et de leurs répartitions

- Conceptions et stack technique

 - MCD/MLD,

 - Documentation des API

- Sécurité et éléments mis en places

- Guide d'installation du/des projets (on part d'un serveur de base, comment on configure pour faire fonctionner le/les projets (commands, configuration, ...))

- Bilan du projet

- Prise de recul et améliorations possibles

Vous y ajouterez, pour la SAE une page avec les différents accès, liens et comptes de démonstration